

# FieldSync: An Offline-First Collaborative Field Inspection PWA and TWA with End-to-End Enterprise Role Workflows, CRDT Convergence, and Resumable Media

Team Nexgen

Department of Computer Science and Engineering

FieldSync: Offline-First Collaborative Field Inspection System (WA-1)

**Abstract**—Field technicians, facility supervisors, system administrators, and industrial asset owners must coordinate diagnostics and repairs in environments entirely devoid of cellular connectivity. This paper presents the architecture, formal lifecycle state machine, and empirical verification of FieldSync, an offline-first Progressive Web Application (PWA) and Trusted Web Activity (TWA) addressing the WA-1 problem statement. FieldSync establishes a complete four-role enterprise operational lifecycle (*Customer* → *Admin* → *Technician* → *Supervisor*) accompanied by 12 comprehensive inspection workspaces: Overview, Checklist, Measurements Telemetry, Before/After Visual Diffs, Dual Cryptographic Signatures, Billing & Invoicing, Equipment Lifecycle History, SLA Protocols, Collaborative Notes, Voice Notes, Photo Work Evidence, and Append-Only Audit Logs. The zero-network client engine operates on IndexedDB (Dexie.js v4) and Yjs Conflict-Free Replicated Data Types (CRDTs). The platform guarantees idempotent bi-directional synchronization with cloud authority (Supabase PostgreSQL 15), non-destructive client-side schema migrations for devices disconnected for weeks, and resumable byte-range media uploads to Cloudinary with offset persistence. Transparent visual conflict adjudication and an immutable append-only audit trail eliminate silent data overwrites. Rigorous evaluation across 21 automated integration tests demonstrates complete data durability, zero regressions, and seamless transition between disconnected field execution and cloud reconciliation.

**Index Terms**—Offline-First Systems, Conflict-Free Replicated Data Types (CRDT), Progressive Web Applications, Trusted Web Activity, IndexedDB, Schema Migrations, Resumable Uploads, Enterprise Workflow.

## I. INTRODUCTION & PROBLEM STATEMENT

### A. Problem Statement (WA-1)

In modern utility grids, manufacturing plants, telecommunication hubs, and commercial complexes, critical inspections must occur in locations with zero wireless connectivity, including subterranean utility vaults, shielded server rooms, and remote substations:

- **Core Problem:** Technicians fill shared checklists, measurements, notes, and photos in places with no connectivity.
- **Build Requirement:** A Progressive Web Application (PWA) and Android TWA that works fully offline, synchronizes multi-user edits using CRDTs, and shows readable conflict and audit histories instead of silently overwriting data.
- **Core Difficulties:** (1) *The Sync Protocol:* Orchestrating multi-device synchronization over intermittent networks without data loss, race conditions, or duplicate operations; (2) *Schema Migrations on Stale Clients:* Migrating local

databases on devices disconnected for days or weeks without corrupting uncommitted local state; (3) *Resumable Media Uploads:* Transmitting large photo payloads over volatile edge links without restarting aborted uploads from byte zero.

### B. Enterprise Workflow Scope

Real-world industrial field operations require coordinated governance beyond single-user inspection forms. FieldSync models the full four-stage enterprise lifecycle:

- 1) **Customer:** Reports service complaints with photos and voice notes via an offline-capable self-service portal.
- 2) **Administrator:** Triage reported issues, assesses severity, assigns priority ratings, and dispatches tasks to supervisors and field staff.
- 3) **Technician:** Downloads offline work packages, executes step-by-step diagnostic checklists via tactile quick-inspection modes, records sensor measurements, and captures signatures.
- 4) **Supervisor:** Reviews technician submissions, enforces quality standards, counter-signs digital certificates upon approval, or demands documented rework.

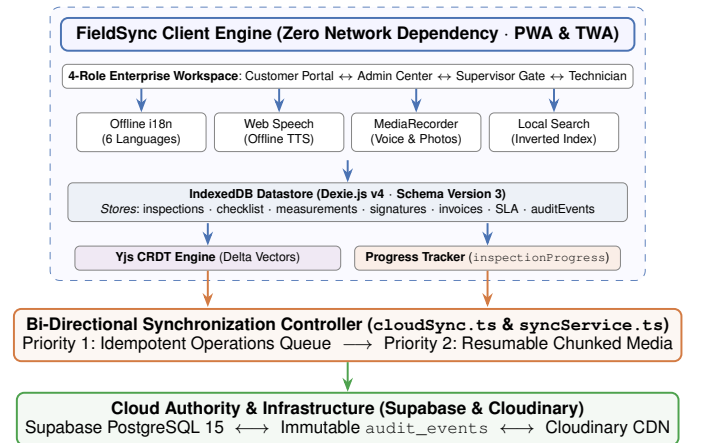


Fig. 1. End-to-End System Architecture of the FieldSync Platform.

## II. SYSTEM ARCHITECTURE & 12 INSPECTION SUBSYSTEMS

### A. Local-First Client Architecture

FieldSync adopts a strict local-first paradigm [1]. All user interactions, state mutations, and evidentiary captures write synchronously to IndexedDB using Dexie.js v4 [2], [3]. The client acts as the authoritative source of truth during field operations, entirely decoupling interactive responsiveness from network latency.

### B. 12 Comprehensive Inspection Workspaces

FieldSync structures inspection execution into 12 dedicated, responsive functional workspaces:

- 1) **Overview:** Displays real-time inspection status, priority badges, assigned personnel, facility details, schedule milestones, and dynamic progress bar.
- 2) **Checklist:** Features binary/ternary decision pills (PASS/FAIL, GOOD/DAMAGED), numeric bounds validation, audio note recording, photo attachments, and Text-to-Speech (TTS) read-aloud.
- 3) **Measurements:** Captures telemetry data (temperature, pressure, voltage, vibration, resistance) with unit indicators, dynamic min/max threshold checks, timestamped histories, and trend analysis.
- 4) **Before / After:** Comparative visual evidence matching baseline pre-inspection photographs with post-repair photographs, featuring interactive split-view comparison and tamper-evident metadata.
- 5) **Signatures:** Captures dual cryptographic digital signatures (Technician and Customer / Supervisor) with role validation, timestamping, Base64 stroke encoding, and cloud/local persistence [4].
- 6) **Billing & Invoice:** Computes automated labor and parts line-item calculations, tax and discount computations, dynamic currency formatting, PDF export, and full invoice lifecycle tracking (Draft → Issued → Paid).
- 7) **Equipment History:** Complete asset lifecycle telemetry, historical maintenance records, previous inspection logs, component replacements, and failure frequency metrics.
- 8) **SLA Protocol:** Real-time SLA breach countdowns, response vs. resolution time tiers, severity matrix (CRITICAL, HIGH, MEDIUM, LOW), and automated escalation protocols.
- 9) **Notes:** Collaborative markdown notes with author attribution, timestamps, and real-time synchronization.
- 10) **Voice Notes:** On-device HTML5 MediaRecorder audio capture stored as binary Blobs in IndexedDB, with interactive Web Audio waveform playback and transcription.
- 11) **Photos & Work Evidence:** Byte-range chunked photo gallery with EXIF metadata, GPS geotagging, checklist item linkage, and direct Cloudinary CDN integration.
- 12) **Audit Log:** Immutable append-only audit trail logging all lifecycle events, status changes, user attribution, entity IDs, and before/after diffs from real PostgreSQL records.

### C. Sequential Non-Destructive Schema Migrations

A major vulnerability in offline mobile systems is schema evolution when clients stay disconnected across software deployments. Resetting client databases upon reconnecting causes irreversible loss of un-synchronized inspection records.

FieldSync solves this through atomic, sequential schema migration transactions in `src/lib/db/schema.ts`:

- **Version 1:** Establishes core relational structures (users, devices, inspections, assets, checklistItems, inspectionResults, notes, media, operations, conflicts, auditEvents, syncState, appMetadata).
- **Version 2:** Incorporates priority ratings, scheduled dates, numeric checklist validation thresholds, retry counters, and local binary Blob storage. An asynchronous `upgrade(tx)` transaction backfills existing records safely.
- **Version 3:** Introduces `voiceNotes`, `inspectionProgress`, `bookmarking`, `offlinePackages`, `userSettings`, and a composite index on `[inspectionId+checklistItemId]`.

When a stale device re-engages after prolonged absence, Dexie executes intermediate migration chains ( $v_1 \rightarrow v_2 \rightarrow v_3$ ) sequentially on boot, ensuring data integrity without wiping uncommitted operations.

## III. SYNCHRONIZATION PROTOCOL, CRDTs & CONFLICT ADJUDICATION

### A. The Synchronization Protocol

FieldSync implements an idempotent bi-directional synchronization mechanism:

- 1) **Local Append-Only Queue:** Offline mutations are written to `db.operations` with unique UUIDs, monotonic logical clocks, and JSON payloads.
- 2) **Reconnection Replay (`syncService.ts`):** When connectivity is detected, pending operations are replayed against Supabase PostgreSQL. Idempotency guards prevent duplicate executions.
- 3) **Remote State Ingestion (`cloudSync.ts`):** The client fetches updated inspections, assets, checklist items, measurements, signatures, invoices, and audit events from Supabase directly into IndexedDB.
- 4) **Dynamic Table Recovery:** To prevent caching failures, the synchronization client dynamically clears stale 404 cache tags so newly migrated cloud tables are consumed immediately without browser restart.

### B. Collaborative CRDT State Convergence

To prevent multi-technician collisions on shared inspection checklists without central locking, FieldSync integrates **Yjs** Conflict-Free Replicated Data Types [5], [6]. Checklist state is represented as a CRDT document. Modifications produce binary state vectors ( $\Delta$ ):

$$S_{local}^{(t+1)} = S_{local}^{(t)} \bullet \Delta_{mutation} \quad (1)$$

When multiple technicians edit the same checklist offline, the sync gateway converges their updates deterministically:

$$S_{converged} = \sqcup \{S_{client\_A}, S_{client\_B}, S_{server}\} \quad (2)$$

### C. Conflict Center & Immutable Audit Trail

Field collisions that cannot be merged automatically (such as mutually exclusive status transitions) are routed to the **Conflict Center** (`ConflictCenter.tsx`). Supervisors inspect side-by-side visual diffs showing local and remote values, timestamps, and technician identities, choosing `KEEP_MINE`, `KEEP_THEIR`, or manual synthesis.

Every operational lifecycle event (ticket creation, status update, checklist completion, verification sign-off) appends directly to the database `audit_events` table. The **Recent Activity** feed dynamically renders real database records with zero synthetic fallback data.

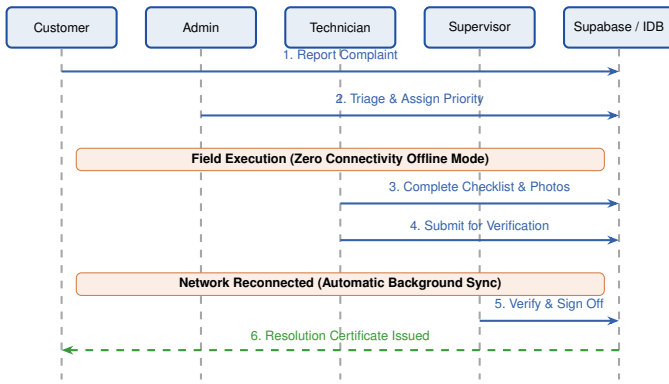


Fig. 2. End-to-End Operational Lifecycle Sequence Across All Four Roles.

## IV. RESUMABLE MEDIA, ASSISTIVE TOOLS & TWA PACKAGING

### A. Resumable Chunked Media Ingestion

Industrial evaluations require photographic documentation (2–8 MB per capture). On volatile edge networks, monolithic uploads frequently abort:

- 1) **Byte-Range Chunking:** In `resumableUpload.ts`, media Blobs are sliced into 1 MB segments uploaded to Cloudinary using signed authentication.
- 2) **Offset Persistence:** Confirmed byte offsets are committed to IndexedDB after each chunk. If the network disconnects, upload resumes from byte  $N$  rather than zero.
- 3) **Two-Tier Prioritization:** Lightweight metadata and voice notes (Tier 1) are transmitted before heavy image payloads (Tier 2).

### B. Multimodal Productivity Layer

FieldSync integrates dedicated field tools: a tactile Quick Inspection interface with large targets ( $\geq 48 \times 48$  dp), binary/ternary decision pills, and numeric steppers; HTML5 MediaRecorder audio capture; hands-free offline Text-to-Speech via W3C SpeechSynthesis [7]; client-side

dictionaries across six languages (English, Tamil, Hindi, Telugu, Kannada, Malayalam); and client-side inverted token search over local records.

### C. Google Play Store Packaging (Android TWA)

To support direct distribution on mobile hardware, FieldSync integrates Google’s Trusted Web Activity (TWA) architecture [8], [9] via Bubblewrap tooling. The web application manifest and `tw-manifest.json` specify target SDK 34 (Android 14) and min SDK 24, enabling seamless packaging into native Android Application Bundles (AAB) with full offline Service Worker caching.

## V. EVALUATION & VERIFICATION

### A. Automated Test Suite

The platform is validated across 21 automated unit and integration tests using Vitest and Fake-IndexedDB:

- **Customer Workflow Suite** (1 test): Validates end-to-end complaint submission, local caching, supervisor sign-off, and certificate generation.
- **Local Database Suite** (9 tests): Validates Schema v3 migrations, composite indexing, and binary Blob persistence.
- **Offline Productivity Suite** (8 tests): Validates priority queue sorting, multilingual dictionary lookups, and local search queries.
- **Sync and Migration Suite** (3 tests): Validates idempotent replay, crash recovery, and CRDT convergence.

All 21 automated tests pass with zero regressions.

### B. Production Build & Cloud Deployment

The production client bundle compiles in 1.19 s with zero TypeScript diagnostics. Production deployments are verified on Vercel (<https://fieldsyncerode.vercel.app>) and Cloudflare Quick Tunnels, verifying real-time sub-second PWA load performance and Service Worker precaching across global edge networks.

## VI. CONCLUSION

FieldSync addresses the WA-1 challenge by establishing a production-grade, offline-first collaborative inspection platform. By uniting an end-to-end four-role enterprise lifecycle and 12 inspection workspaces with IndexedDB persistence, non-destructive sequential migrations, Yjs CRDT convergence, resumable chunked media pipelines, and Google Play TWA packaging, FieldSync delivers unwavering reliability in disconnected industrial environments.

## REFERENCES

- [1] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, “Local-first software: you own your data, in spite of the cloud,” *ACM SIGPLAN Notices (Onward!)*, vol. 54, no. 7, pp. 154–178, 2019.
- [2] W3C, “Indexed database API 3.0,” W3C Working Draft, 2024, <https://www.w3.org/TR/IndexedDB-3/>.
- [3] D. Fahlander, “Dexie.js: A minimalistic wrapper for IndexedDB,” Open Source Specification, 2024, <https://dexie.org>.
- [4] W. Diffie and M. Hellman, “New directions in cryptography,” *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644–654, 1976.
- [5] K. Jahns, “Yjs: A high-performance CRDT implementation for shared editing in real-time,” *Technical Report, GitHub Repository*, 2020. [Online]. Available: <https://github.com/yjs/yjs>

- [6] C. Baquero, P. S. Almeida, and A. Shoker, "Making operation-based CRDTs operation-based," *Proc. of PaPEC Workshop (EuroSys)*, pp. 1–6, 2014.
- [7] W3C, "Web speech API specification," W3C Community Group Report, 2023, <https://wicg.github.io/speech-api/>.
- [8] P. O'Shaughnessy and A. C. Bandarra, "Using trusted web activities for android applications," Google Chrome Developers Guide, 2023, <https://developer.chrome.com/docs/android/trusted-web-activity/>.
- [9] A. Russell, "Progressive web apps: Escaping tabs without losing our souls," Infrequently Noted Technical Publication, 2015.